

Orchestrating Intelligence: A Comprehensive Survey on Tool Learning and Function Calling for Large Language Model Agents

1 Introduction

The evolution of Large Language Models (LLMs) marks a fundamental paradigm shift from static, parametric knowledge repositories to dynamic, agentic systems capable of active environmental interaction. While foundational models exhibit remarkable proficiency in natural language understanding and generation, their reliance on fixed training data inherently limits their capacity to access real-time information, execute precise computational operations, or interact with external digital ecosystems [1]. This survey posits that the integration of tool learning and function calling represents the critical mechanism for transcending these limitations, effectively transforming LLMs from passive text generators into autonomous agents that can perceive, reason, and act within complex environments [2]. By augmenting models with external tools, we enable a synergy where the LLM provides high-level planning and semantic reasoning, while specialized tools handle deterministic execution and data retrieval, thereby mitigating hallucinations and enhancing factual grounding [3].

To establish a unified terminology for this review, we formally define **tool learning** as the process by which an agent acquires the capability to select, parameterize, and invoke external functions to solve tasks beyond its intrinsic capabilities [4]. Distinct from mere retrieval, **function calling** constitutes the syntactic and semantic protocol through which the model translates natural language intents into structured, executable commands, often constrained by schemas such as JSON or executable code [5]. Furthermore, **agentic behavior** is characterized by the closed-loop interaction where the agent observes environmental feedback, reflects on execution outcomes, and iteratively refines its strategy, a process essential for handling long-horizon tasks [6].

The necessity of this tool augmentation is driven by three primary motivations. First, it addresses the staleness of parametric knowledge, allowing agents to access ephemeral data through APIs, a capability demonstrated in domains ranging from scientific discovery to financial analysis [7]. Second, it enhances computational precision; while LLMs struggle with exact arithmetic or logical deduction, delegating these sub-tasks to calculators or code interpreters ensures reliability [8]. Third, it expands the action space, enabling agents to manipulate software interfaces, control robotic hardware, or manage database states, effectively bridging the gap between digital cognition and physical or virtual actuation [9]. Recent frameworks like [10] and [3] illustrate how embedding tool representations or employing self-play mechanisms can significantly boost performance without necessitating massive model scaling.

This survey systematically reviews the landscape of tool learning, organized by architectural paradigms, cognitive strategies, learning methodologies, and evaluation ecosystems. We begin by analyzing structural designs, contrasting monolithic fine-tuned models with modular orchestration patterns that decouple planning from execution [11]. Subsequently, we explore advanced cognitive strategies for hierarchical task decomposition and self-reflection, which are vital for navigating complex solution spaces [12]. We then examine learning frameworks, from supervised fine-tuning on curated trajectories to reinforcement learning via environmental feedback [13]. Finally, we assess the current evaluation benchmarks and safety protocols required for deploying robust agents in high-stakes scenarios [14]. By synthesizing these dimensions, this work aims to provide a comprehensive roadmap for advancing the field toward truly autonomous, reliable, and scalable intelligent agents.

2 Architectural Paradigms and Mechanism Design for Tool Integration

2.1 Structural Architectures and Agent Orchestration Patterns

The architectural landscape of tool-augmented Large Language Model (LLM) agents is characterized by a fundamental dichotomy between monolithic end-to-end systems and modular, orchestrated ecosystems. This spectrum dictates how agents manage state, delegate authority, and navigate the complexity of external tool interaction. Monolithic architectures typically rely on a single, highly capable LLM to internalize tool logic through extensive fine-tuning, treating tool invocation as a native generation task. While approaches like [15] demonstrate that hybrid instruction tuning can enhance generalized agent abilities without compromising core linguistic competence, such unified models often struggle with the combinatorial explosion of action spaces in complex, multi-step domains. The limitations of single-agent paradigms become particularly acute when facing dynamic environments requiring distinct cognitive capabilities for planning, execution, and reflection simultaneously.

To mitigate these constraints, recent research has pivoted toward modular frameworks that decouple these functions into specialized components. A prominent manifestation of this shift is the Multi-Agent Orchestration pattern, where distinct agents assume specific roles within a collaborative workflow. For instance, [16] introduces a controller-managed architecture where labor agents specialize in specific action types, thereby optimizing the compatibility between agent roles and LLM backbones. Similarly, [11] modularizes the tool learning workflow into Grounding, Execution, and Observing agents, employing iterative calibration to adapt to environmental feedback more effectively than monolithic counterparts. This decomposition allows for the utilization of smaller, specialized models for specific sub-tasks, addressing the finding that small LLMs often underperform as generalist tool learners when forced to handle all cognitive loads simultaneously [17]. Furthermore, frameworks like [18] extend this concept by dynamically generating specialized agent teams based on task content, incorporating observer roles to reflect on and refine collective strategies.

Beyond role specialization, structural innovation extends to the representation of agent workflows as computational graphs. The paradigm of “Language Agents as Optimizable Graphs” posits that agents can be modeled as nodes and edges within a graph structure, where automatic optimizers refine both node-level prompts and edge-level connectivity to improve orchestration [19]. This graph-based perspective facilitates hierarchical planning and deterministic path finding, as seen in systems that employ tree-search algorithms like A* to navigate expansive action spaces efficiently [20]. Such structures enable explicit management of information flow and state dependencies, contrasting sharply with the implicit state tracking of linear conversational agents.

The trade-offs between these architectures are nuanced. Monolithic systems offer lower latency and simpler deployment but lack the robustness and interpretability of modular systems. Conversely, multi-agent and graph-based architectures provide superior scalability and fault isolation—critical for high-stakes applications like software engineering [21]—but incur higher communication overhead and coordination complexity. Emerging trends suggest a convergence towards hybrid models, such as [22], which provides a lightweight infrastructure to flexibly switch between single-agent reasoning and multi-agent collaboration based on task difficulty. Future architectural designs must therefore balance the cognitive depth of centralized controllers with the distributed resilience of specialized swarms, potentially leveraging neuro-symbolic hybrids to enforce structural constraints on agent interactions while maintaining the semantic flexibility of LLMs.

2.2 Interface Design Standards and Syntax-Constrained Decoding

While the preceding section established the architectural foundations for orchestrating agents, the efficacy of these structures ultimately hinges on the interface mechanisms that translate semantic intent into executable actions. This interface constitutes the critical boundary where abstract reasoning meets concrete tool invocation. Early paradigms relied on unstructured prompt engineering, where models generated free-form text containing function calls—an approach prone to syntactic fragility and hallucination. Contemporary standards have therefore shifted toward rigorous, schema-based interaction protocols that enforce structural integrity. A primary evolution involves the standardization of function descriptions using formats like JSON Schema or OpenAPI; however, the verbosity of such documentation often exhausts context windows, creating a bottleneck that architectural design alone cannot solve. To mitigate this, frameworks like [23] transform diverse, lengthy documentation into unified, concise tool instructions, purifying essential information to reduce token overhead while maintaining semantic fidelity. Similarly, [24] demonstrates that coupling LLMs with document retrievers allows for dynamic adaptation to API changes, significantly reducing hallucinations in argument generation by grounding the model in up-to-date technical specifications.

Beyond prompt optimization and retrieval, the decoding process itself has become a focal point for ensuring reliability within these interfaces. Traditional autoregressive generation offers no guarantees that the output will conform to the strict grammar of an API call. Syntax-constrained decoding addresses this by integrating finite-state machines (FSMs) or context-free grammars directly into the sampling loop. Techniques such as those proposed in [25] utilize conditional masking to restrict the model’s output distribution to valid tokens at each step, ensuring zero syntactic errors and type conformity without necessitating extensive fine-tuning. This approach effectively decouples the model’s reasoning capability from its generative precision, allowing smaller on-device models to outperform larger generalist models in specific function-calling tasks. Furthermore, [26] extends this concept to code repair by synergizing LLMs with completion engines that prune infeasible tokens and proactively complete structures based on static analysis, thereby ensuring that generated patches are not only semantically relevant but also statically valid.

A transformative trend in interface design is the paradigm shift from rigid, predefined JSON schemas to executable code as a unified action space. The [27] framework argues that representing actions as executable Python code rather than structured text enables dynamic composition, complex control flow, and self-debugging capabilities that static schemas cannot support. By leveraging a Python interpreter, agents can iteratively revise actions based on execution feedback, creating a closed-loop interaction that mimics human developer workflows. This flexibility is complemented by visual and low-code interaction layers, as seen in [28] and [29], which allow humans to visually orchestrate chains of LLM calls or adapt system prompts within design environments. These tools bridge the gap between abstract agentic logic and concrete implementation, enhancing transparency and controllability.

However, trade-offs persist between the flexibility of code-based interfaces and the safety guarantees of constrained decoding. While executable code offers superior expressiveness, it introduces significant security risks and requires robust sandboxing. Conversely, schema-constrained approaches provide strong safety guarantees but may limit the agent’s ability to handle novel or composite tasks. Future research must therefore focus on hybrid architectures that combine the dynamic reasoning of code-based agents with the rigorous type safety of grammar-guided decoding. Additionally, as multimodal agents emerge, interface standards must evolve to accommodate non-textual

grounding, ensuring that visual and auditory inputs are correctly mapped to tool parameters, a challenge beginning to be addressed by systems like [30]. Yet, even with robust interface protocols, the sheer scale of modern API ecosystems introduces a prior bottleneck: the efficient discovery and selection of relevant tools within constrained context windows, a challenge that necessitates advanced retrieval mechanisms as explored in the following section.

2.3 Scalable Tool Discovery and Semantic Retrieval Mechanisms

As the ecosystem of available APIs expands into the tens of thousands, the fundamental bottleneck for Large Language Model (LLM) agents shifts from execution capability to the efficient discovery and selection of relevant tools within constrained context windows. Loading comprehensive documentation for massive repositories is computationally prohibitive, necessitating advanced retrieval mechanisms that bridge the semantic gap between ambiguous user intents and precise function signatures. Contemporary approaches increasingly rely on dense retrieval and intent-aware matching to navigate this complexity. For instance, frameworks like AnyTool employ hierarchical retrieval structures to filter candidates from over 16,000 APIs, significantly outperforming flat retrieval baselines by organizing tools into manageable semantic clusters [31]. Similarly, GeckOpt demonstrates that identifying high-level user intent at runtime can dynamically prune the candidate toolset, reducing token consumption by nearly 25% while maintaining selection accuracy [32].

However, static retrieval often fails to resolve nuances where multiple tools possess similar functionalities or where documentation is verbose and redundant. To address this, recent research emphasizes the co-optimization of retrievers and generators through iterative feedback loops. The ToolSandbox benchmark highlights that stateful, conversational evaluation is critical for assessing an agent’s ability to refine tool selection based on intermediate execution feedback, rather than relying on single-turn predictions [33]. This aligns with the principles of self-verification, where models like TOOLVERIFIER generate contrastive questions to distinguish between close candidate tools during the selection phase, achieving substantial improvements in generalization to unseen APIs [34]. Furthermore, the integration of multi-modal encoders, as seen in MLLM-Tool, allows agents to leverage visual or auditory cues to disambiguate tool selection when textual descriptions alone are insufficient, particularly in scenarios involving synonymous functions [30].

A paradigm shift is also occurring from passive retrieval to active tool creation and abstraction. Rather than being limited to pre-defined repositories, agents can now autonomously synthesize, validate, and catalog new tool snippets from code solutions, effectively expanding the action space dynamically. The CRAFT framework exemplifies this by creating task-specific toolsets where code solutions are abstracted into reusable snippets, enabling plug-and-play adaptation to unseen domains without fine-tuning [35]. This complements efforts to streamline tool interfaces; EASYTOOL transforms lengthy, heterogeneous documentation into unified, concise instructions, thereby enhancing the signal-to-noise ratio for retrieval models and improving downstream task performance [23]. Additionally, the automatic generation of verifiable datasets via pipelines like APIGen ensures that retrieval mechanisms are trained on high-quality, diverse function-calling examples, mitigating the scarcity of reliable training data for rare API combinations [36].

Despite these advancements, significant challenges persist regarding robustness against noise and the scalability of evaluation. Benchmarks such as RoTBench reveal that even state-of-the-art models suffer drastic performance degradation when confronted with slight perturbations in tool descriptions or environmental noise, indicating a fragility in current semantic matching strategies [37]. Moreover, the transition from synthetic benchmarks to real-world complexity requires evalu-

ation frameworks that account for implicit state dependencies and long-horizon planning, as static retrieval metrics often fail to capture the dynamic nature of tool interaction [38]. Future research must therefore focus on developing neuro-symbolic retrieval architectures that combine the semantic flexibility of LLMs with the logical rigor of symbolic search, ensuring that tool discovery remains both scalable and semantically grounded in increasingly complex operational environments.

2.4 Parameter Grounding and Argument Extraction Protocols

Once the optimal tool is selected, the challenge shifts to the critical interface of parameter grounding and argument extraction, where abstract natural language intents must be transduced into concrete, typed invocations capable of altering environmental states. This process transcends mere syntactic parsing; it demands a robust mechanism for resolving semantic ambiguity, inferring implicit constraints, and ensuring logical consistency with the dynamic context of the agent’s operation. The fundamental challenge lies in bridging the gap between the fluidity of human instruction and the rigid schema requirements of external APIs, a discrepancy that often leads to execution failures when parameters are underspecified or contextually misaligned.

A primary avenue of research focuses on mapping unstructured natural language to structured argument spaces through advanced semantic parsing and constraint satisfaction. Approaches such as [39] demonstrate that while Large Language Models (LLMs) excel at leveraging commonsense knowledge to fill gaps in under-specified goals, they remain susceptible to errors in numerical and spatial reasoning without explicit grounding mechanisms. To mitigate this, recent frameworks employ syntax-constrained decoding and finite-state machines to enforce type conformity during generation, ensuring that extracted arguments strictly adhere to predefined tool signatures. Furthermore, the [40] framework introduces a decoupled strategy where LLMs first generate reasoning chains with abstract placeholders, subsequently reifying these into specific tool arguments. This separation allows for parallel decoding and tool invocation, significantly enhancing efficiency while maintaining robustness against domain shifts in parameter values.

Beyond static mapping, effective grounding necessitates active interaction to resolve uncertainty, particularly when user instructions lack necessary detail. The paradigm of “asking for help” is formalized in [41], which utilizes conformal prediction to quantify uncertainty in parameter slots, triggering clarification queries only when confidence falls below a statistical threshold. This aligns with the interactive task learning proposed in [42], where agents collaboratively resolve vague concepts through multi-turn dialogue and visual demonstrations, effectively refining parameter boundaries in real-time. Such environment-grounded assumption repair mechanisms are crucial for preventing hallucinated arguments that could lead to irreversible actions or safety violations.

The integration of multimodal inputs further complicates argument extraction, requiring agents to ground textual references within visual or auditory contexts. Systems like [30] address this by incorporating multimodal encoders that allow agents to perceive visual-grounded instructions, thereby resolving referential ambiguities that text-only models cannot handle. Similarly, [43] argues for inherently grounded planners that integrate perceptual data directly into the reasoning loop, enabling precise parameter extraction for robotic manipulation tasks based on object attributes and spatial layouts. In complex software engineering contexts, [44] demonstrates how agents can dynamically gather bug context and validate fix arguments through tool interaction, illustrating a closed-loop grounding process where parameter validity is verified against compiler feedback rather than static schemas.

Despite these advances, significant challenges persist regarding robustness against prompt perturba-

tions and the scalability of grounding mechanisms across diverse domains. Current methods often struggle with long-horizon tasks where parameter dependencies evolve over time, a limitation highlighted by [33], which identifies state dependency and canonicalization as major failure points for state-of-the-art models. Future research must therefore focus on developing neuro-symbolic hybrids that combine the flexibility of LLMs with the rigor of formal verification, ensuring that parameter grounding remains both semantically rich and logically sound in increasingly autonomous agentic ecosystems.

3 Cognitive Strategies for Planning, Reasoning, and Execution Control

3.1 Hierarchical Task Decomposition and Semantic Planning

Hierarchical task decomposition serves as the foundational cognitive mechanism for translating ambiguous, high-level user intents into structured, executable sequences of tool invocations. The transition from monolithic prompting to modular planning architectures represents a critical evolution in agent design, addressing the inherent limitations of large language models (LLMs) in handling long-horizon dependencies and complex logical constraints. Early approaches relying on direct plan generation often suffer from “planning hallucinations,” where models synthesize invalid tool sequences due to a lack of explicit action knowledge. To mitigate this, recent frameworks integrate explicit action knowledge bases and affordance models to constrain the search space during plan generation. For instance, [45] employs an action knowledge base alongside a knowledgeable self-learning strategy to guide planning trajectories, effectively reducing hallucinations by enforcing adherence to valid action paths. Similarly, [46] introduces a parametric World Knowledge Model (WKM) that synthesizes prior task knowledge and dynamic state information, steering agents away from blind trial-and-error toward grounded, feasible planning.

A prominent trend in this domain is the synthesis of LLM-based semantic understanding with classical symbolic planners, creating neuro-symbolic hybrid architectures that guarantee logical consistency. While LLMs excel at commonsense reasoning and intent parsing, they frequently fail to generate optimal or even feasible plans in isolation, as evidenced by low success rates in classical planning benchmarks [47; 48; 49]. Consequently, frameworks like [50] and [51] leverage LLMs as translators to convert natural language into formal planning domain definition languages (e.g., PDDL), delegating the actual search and optimization to sound, external symbolic solvers. This division of labor allows the LLM to function as a heuristic generator or interface layer, while the symbolic backend ensures optimality and correctness. Further enhancing this paradigm, [52] enables symbolic planners to operate in open-world settings by using LLMs to incrementally build knowledge bases when execution errors reveal gaps in pre-defined domain models.

Beyond hybridization, dynamic goal refinement and adaptive sub-task granularity are essential for managing task complexity. Static decomposition strategies often fail when sub-tasks exceed the executor’s capabilities or when environmental feedback necessitates replanning. The [29] framework addresses this by implementing an “as-needed” decomposition strategy, recursively breaking down sub-tasks only when the LLM executor cannot proceed, thereby dynamically aligning plan granularity with model capabilities. Complementing this, [53] introduces a dynamic solution tree approach that clusters tools by functionality, allowing agents to re-select alternative tools within a kit upon failure without restarting the entire planning process. Furthermore, the integration of visual grounding has become increasingly vital; [43] demonstrates that incorporating perceptual data directly into the planning loop enables agents to reason about spatial layouts and object attributes, significantly improving plan executability in physical environments.

Despite these advances, challenges remain in balancing the computational cost of search-based methods with the latency requirements of real-time interaction. While tree-search algorithms like Monte Carlo Tree Search can enhance reasoning [54; 55], their efficacy is heavily contingent on the accuracy of the discriminator or value function, which current LLMs may not reliably provide [56]. Future research must therefore focus on developing lightweight, learned value functions that can guide hierarchical decomposition efficiently, as well as robust mechanisms for integrating multi-modal feedback to ensure plans remain grounded in dynamic environmental states.

3.2 Advanced Search Strategies and Solution Space Exploration

Building upon the hierarchical structures and dynamic granularity established in the previous section, navigating the resulting combinatorial explosion of potential tool interactions requires agents to transcend greedy, single-path execution in favor of sophisticated search algorithms that explicitly model the solution space. This subsection examines how tree-based search, Monte Carlo methods, and solution clustering enable agents to evaluate prospective trajectories, balancing the trade-off between the exploration of novel strategies and the exploitation of verified paths. While hierarchical decomposition defines the “what” and “how” of task breakdown, search algorithms determine the optimal trajectory through the generated solution tree, a critical step before agents can engage in the self-correction mechanisms discussed later.

The foundational shift from linear Chain-of-Thought reasoning to structured search is exemplified by frameworks that repurpose Large Language Models (LLMs) as both world models and reasoning agents within a planning algorithm. [55] introduces Reasoning via Planning (RAP), which employs Monte Carlo Tree Search (MCTS) to incrementally build a reasoning tree, guiding the agent toward high-reward paths through strategic exploration. Similarly, [57] proposes LATS, a general framework synergizing LLMs with MCTS to unify planning, acting, and reasoning, demonstrating superior performance in programming and web navigation by leveraging environmental feedback for value estimation. These approaches directly address the inherent non-determinism and hallucination risks in code generation and multi-step planning, as highlighted by empirical studies showing that static prompting often fails to guarantee consistent outputs [58].

However, the efficacy of such search methods is contingent upon the quality of the discriminator or value function used to prune the search tree, a limitation foreshadowed in the discussion of planning bottlenecks. [56] provides a critical analysis, revealing that advanced planning methods like tree search require discriminators with at least 90% accuracy to outperform simpler re-ranking strategies; without such precision, the computational overhead yields negligible gains. To mitigate this dependency on perfect discrimination, [59] presents TS-LLM, an AlphaZero-like framework that leverages a learned value function to guide decoding and training iteratively, proving adaptable to tasks with varying search depths up to 64 steps. Furthermore, [60] demonstrates that best-first tree search operating within the actual environment space can significantly boost success rates on realistic web tasks, scaling performance with increased test-time compute.

Beyond explicit tree search, solution space exploration also benefits from clustering and abstraction mechanisms that reduce redundancy and enhance generalization. [61] introduces UltraInteract, a dataset constructing preference trees with diverse planning strategies, enabling models to learn from multi-turn interaction trajectories and critiques rather than isolated successful paths. Complementing this, [1] emphasizes the necessity of dynamic plan adjustment through reasoning, while [54] argues that using LLMs as world models within MCTS provides a commonsense prior that vastly improves search efficiency compared to policy-only approaches. The integration of these

search strategies with self-reflection loops, as seen in [62], allows agents to iteratively refine their policies without additional annotations, effectively closing the loop between search, execution, and learning, and setting the stage for the reactive correction paradigms detailed in the next section.

Despite these advances, challenges remain in optimizing the exploration-exploitation trade-off for cost-efficiency, particularly given the latency constraints of real-time interaction. [63] suggests that agents utilizing uncertainty estimation, such as double Thompson sampling, can gather high-quality feedback with fewer queries, addressing the prohibitive costs associated with extensive search in large action spaces. Future research must therefore focus on developing lightweight, learned heuristics that maintain the robustness of exhaustive search while adhering to practical deployment limits. Ultimately, while search algorithms optimize trajectory selection, the resilience of the agent when these trajectories fail depends on the capacity for self-reflection and iterative correction, a topic explored in the following subsection.

3.3 Self-Reflection, Error Recovery, and Iterative Correction Loops

Self-reflection and iterative correction constitute the critical cognitive mechanisms that transform static language models into resilient, autonomous agents capable of navigating complex, dynamic environments. Unlike passive generators, advanced agents must possess the capacity to monitor execution trajectories, diagnose discrepancies between intended and actual outcomes, and autonomously initiate repair loops. This subsection analyzes the architectural patterns and algorithmic strategies that enable such self-correction, emphasizing the shift from single-pass generation to closed-loop refinement driven by environmental feedback.

The foundation of robust error recovery lies in the tight integration of reasoning and acting, where agents interleave thought traces with tool invocations to handle exceptions dynamically. The Re-Act paradigm [64] exemplifies this by allowing models to update action plans based on observations, effectively mitigating hallucination and error propagation through active environment interaction. Building on this, adaptive planning frameworks like AdaPlanner [65] introduce mechanisms for both in-plan and out-of-plan refinement, enabling agents to modify static plans in response to real-time feedback, thereby significantly reducing sample complexity in sequential decision-making tasks. Furthermore, the RAFA framework [66] formalizes this process within a Bayesian adaptive Markov decision process, theoretically guaranteeing regret bounds by continuously replanning future trajectories based on accumulated memory buffers.

A pivotal challenge in self-correction is the interpretation of unstructured or technical feedback, such as compiler stack traces or API error codes. In software engineering domains, agents like RepairAgent [44] and SWE-agent [67] leverage specialized agent-computer interfaces to parse execution failures and autonomously navigate repositories for context-aware patching. These systems often employ a “synthesize, execute, debug” cycle, where the agent iteratively refines code until unit tests pass [68]. To enhance the efficacy of these loops, recent work integrates verification modules directly into the generation process. For instance, LEVER [69] trains verifiers to rerank sampled programs based on execution results, while Repilot [26] synergizes LLMs with completion engines to prune infeasible tokens during autoregressive generation, ensuring syntactic validity before execution.

Beyond immediate error repair, sophisticated agents utilize multi-perspective reflection and policy-level optimization to prevent recurrent failures. Approaches such as Retroformer [70] employ policy gradient methods to tune prompts retrospectively, summarizing root causes of failure to optimize future action policies. Similarly, Agent-Pro [71] distinguishes itself by reflecting on entire belief

trajectories rather than isolated actions, utilizing depth-first search to evolve irrational beliefs into robust policies. The efficacy of these loops is further amplified by learning from negative examples; frameworks like TRICE [72] and ETO [73] demonstrate that incorporating failed trajectories into contrastive learning or fine-tuning datasets significantly improves an agent’s ability to discern when and how to use tools, avoiding the pitfall of indiscriminate tool invocation. Moreover, self-improvement loops can be bootstrapped without external supervision, as seen in AlphaLLM [62], which integrates Monte Carlo Tree Search with critic models to refine reasoning capabilities autonomously.

Despite these advancements, challenges remain in balancing the computational cost of iterative loops with the marginal gains in accuracy. While methods like CigaR [74] optimize token usage during repair, the risk of infinite correction loops or “delusional” self-affirmation persists [75]. Future research must therefore focus on developing concise, high-fidelity feedback signals and meta-cognitive controllers that can dynamically terminate reflection processes when diminishing returns are detected, ensuring that self-correction enhances rather than hinders agent efficiency.

3.4 Context Management, State Consistency, and Long-Horizon Coherence

Directly addressing the challenge of hallucinated environmental conditions highlighted previously, maintaining coherent state representations and managing context windows across extended multi-turn interactions constitutes a fundamental bottleneck in deploying Large Language Model (LLM) agents for long-horizon tasks. Unlike static question-answering scenarios, agentic workflows involve dynamic environmental shifts where the validity of subsequent actions depends critically on the outcomes of preceding tool invocations. A primary challenge lies in tracking implicit state dependencies; agents must recognize that a tool call does not merely return data but alters the environment’s configuration. The [33] benchmark explicitly highlights this difficulty, noting that current models frequently fail to maintain state consistency when tools possess implicit dependencies or require canonicalization of intermediate results. Without explicit state tracking mechanisms, agents risk hallucinating environmental conditions, leading to cascading execution errors in complex workflows.

To mitigate context window overflow and preserve critical historical information, researchers have developed sophisticated memory augmentation and compression strategies. The [76] framework introduces a mechanism where agents induce reusable workflows from past experiences, selectively retrieving high-level procedural knowledge rather than raw interaction logs to guide future actions. This approach substantially improves success rates on long-horizon web navigation tasks by reducing the cognitive load on the model. Similarly, [77] proposes a Retrieval-Augmented Planning architecture that dynamically leverages contextual memory, allowing agents to reflect on past experiences relevant to the current situation. This is particularly effective in multimodal environments where maintaining a coherent history of visual and textual observations is essential for grounded decision-making. Furthermore, [78] demonstrates that agents can autonomously gather experiences and extract insights in natural language, creating a compact memory store that enhances performance without requiring parametric updates. These methods collectively suggest a shift from brute-force context retention to semantic summarization and hierarchical retrieval.

Beyond passive memory retention, advanced agents employ active environmental learning to construct internal rule sets or “manuals” that ensure consistency in novel domains. The [79] agent exemplifies this by building a Python program representing its world model through code synthesis, effectively externalizing state management into an executable script that evolves with environmental interactions. This model-based approach offers superior sample efficiency compared to purely

reactive agents. Complementing this, [80] extracts state-aware guidelines from offline experiences, providing conditional natural language rules that help agents navigate states where pre-trained knowledge is insufficient. Such mechanisms allow agents to adapt to dynamic environments by updating their internal logic rather than relying solely on transient context windows. Additionally, [81] underscores the importance of simulated trial and error, where agents use “imagination” to anticipate state changes and refine their memory of tool behaviors before real-world execution, thereby reducing the risk of incoherent long-term planning.

Despite these advancements, significant trade-offs remain between the granularity of state representation and computational efficiency. While explicit state modeling via code [79] or workflow induction [76] enhances robustness, it introduces latency and complexity in the reasoning loop. Future research must address the integration of neuro-symbolic approaches that combine the flexibility of LLMs with the rigorous state guarantees of formal methods. Moreover, as agents operate in increasingly distributed multi-agent systems, developing standardized protocols for shared state consistency and decentralized context management will be paramount to scaling agentic intelligence beyond single-turn interactions.

4 Learning Methodologies and Skill Acquisition Frameworks

4.1 Supervised Fine-Tuning and Curated Trajectory Learning

Supervised Fine-Tuning (SFT) on curated trajectories represents the foundational paradigm for imbuing Large Language Models (LLMs) with robust tool-use capabilities, effectively bridging the gap between static parametric knowledge and dynamic environmental interaction. This approach posits that models can internalize the syntax of function calling, argument grounding, and multi-step reasoning by learning from high-quality, annotated datasets, thereby reducing the reliance on computationally expensive real-time exploration during the initial acquisition phase. A critical challenge in this domain is the scarcity of diverse, human-annotated tool-use data, which has spurred the development of automated corpus construction pipelines. Frameworks such as [82] and [36] leverage multi-agent simulations and LLM-driven self-instruction to generate scalable, synthetic trajectories covering thousands of real-world APIs. These methods create diverse training corpora that include rare edge cases and complex API combinations, enabling compact models to achieve generalized tool-use proficiency comparable to much larger counterparts without extensive human labeling.

Beyond mere data scaling, recent advancements focus on the structural quality of reasoning chains within these trajectories. The [40] methodology introduces a decoupling mechanism where models are trained to first generate abstract reasoning chains with placeholders, subsequently reifying them with specific tool calls. This separation of planning from execution enhances generalization across different toolsets and significantly reduces inference latency by allowing parallel decoding and tool invocation. Similarly, [83] enhances abstraction abilities through explanation tuning, forcing models to capture underlying rationales rather than superficial pattern matching. However, the efficacy of SFT is heavily dependent on the fidelity of the training data; naive fine-tuning on successful trajectories alone can lead to compounding errors when agents encounter novel distribution shifts. To mitigate this, frameworks like [73] and [84] advocate for learning from both successful and failed exploration paths. By constructing contrastive trajectory pairs from agent interactions, these approaches enable Direct Preference Optimization (DPO) to refine policy decisions, correcting erroneous tool selection patterns and improving robustness against execution failures.

The architectural design of the action space also plays a pivotal role in SFT efficacy. While tradi-

tional approaches rely on structured JSON schemas, emerging evidence suggests that consolidating actions into executable code spaces yields superior performance. [27] demonstrates that training models to generate executable Python code as a unified action space allows for dynamic composition of tools and self-debugging, outperforming rigid JSON-based calling formats by up to 20% in success rates. Furthermore, modular strategies such as those proposed in [17] decompose the learning objective into distinct planner, caller, and summarizer roles, allowing smaller models to specialize and collaborate, thereby overcoming inherent capacity limitations in single-model architectures. Despite these advances, SFT faces limitations regarding scalability and the “static” nature of learned policies, which may struggle with dynamic environments not represented in the training set. Future directions must therefore integrate SFT with interactive reinforcement learning loops, leveraging curated trajectories as strong priors while enabling continuous adaptation through environmental feedback, ensuring agents remain effective in evolving real-world scenarios.

4.2 Reinforcement Learning and Interactive Policy Optimization

Building upon the strong priors established by Supervised Fine-Tuning (SFT) but addressing its inherent limitations regarding static policies and distribution shifts, Reinforcement Learning (RL) and interactive policy optimization represent a paradigmatic shift toward dynamic, environment-grounded skill acquisition for Large Language Model (LLM) agents. While SFT relies on curated trajectories, interactive paradigms enable agents to refine tool invocation policies through trial, error, and direct feedback signals, thereby overcoming the scarcity of high-quality demonstration data and enhancing adaptability to novel environments. A fundamental challenge in this domain is the sparsity of reward signals in long-horizon tasks, where success is often binary and delayed. To mitigate this, recent frameworks have explored dense reward shaping and intrinsic motivation. For instance, [85] leverages LLMs to generate intrinsic rewards by evaluating action captions, guiding agents toward human-aligned behaviors in open-ended environments like NetHack without explicit environmental rewards. Similarly, [86] utilizes LLM-suggested goals to shape exploration, ensuring agents visit semantically meaningful states rather than merely novel ones, thus improving sample efficiency in downstream tasks.

Beyond reward engineering, the integration of symbolic verification and execution feedback has emerged as a critical mechanism for providing dense, verifiable learning signals, directly addressing the error propagation issues noted in static training. Unlike black-box reward models, executable environments offer precise error logs and state transitions that can be directly incorporated into the loss function. [87] exemplifies this by employing LLMs to evolve reward code itself, achieving human-level performance in complex robotic manipulation tasks such as pen spinning by optimizing reward functions through evolutionary strategies. Furthermore, [88] demonstrates that incorporating execution feedback into RL training loops significantly enhances an agent’s ability to self-correct, outperforming pure supervised baselines by learning from both successful and failed trajectories. This aligns with the principles of [73], which utilizes contrastive learning on pairs of successful and failed exploration trajectories to optimize policies via Direct Preference Optimization (DPO), effectively turning environmental failures into valuable training data and bridging the gap between static SFT and fully interactive learning.

To manage the complexity of long-horizon planning inherent in real-world tool use, hierarchical and hybrid architectures have been developed to decouple high-level semantic reasoning from low-level control optimization. In these frameworks, the LLM often acts as a high-level planner or reward designer, while a specialized RL agent handles fine-grained execution. [89] bridges this gap by using LLM-generated abstract plans to guide motion planning and RL policies, enabling the solution of

multi-stage robotic tasks from scratch without pre-defined skill libraries. Similarly, [90] posits that LLMs are best suited for defining reward parameters rather than low-level actions, allowing real-time optimizers to synthesize diverse skills based on natural language instructions. This division of labor is also evident in [91], where a planner LLM issues commands to an actor trained via RL, facilitated by a reporter module that grounds the planner in environmental observations.

Despite these advances, significant trade-offs remain regarding sample efficiency and the stability of online fine-tuning, necessitating careful architectural choices to preserve general knowledge. Methods like [92] propose parameter-efficient architectures, such as sharing frozen LLM backbones with Low-Rank Adaptation (LoRA) updates for actor-critic networks, to prevent catastrophic forgetting while adapting to specific environments. Moreover, the reliability of self-improvement loops depends heavily on the fidelity of the feedback mechanism; without robust verification, agents risk reinforcing hallucinated behaviors. Future research must therefore focus on developing more robust verifiers and hybrid neuro-symbolic frameworks that combine the generalization capabilities of LLMs with the rigorous convergence guarantees of traditional RL. While RL provides a powerful mechanism for continuous adaptation, the computational costs and instability of online updates motivate the exploration of alternative paradigms like In-Context Learning and Parameter-Efficient Fine-Tuning, which offer flexible adaptation without full-model retraining, as discussed in the following section.

4.3 In-Context Learning and Parameter-Efficient Adaptation

In-Context Learning (ICL) and Parameter-Efficient Fine-Tuning (PEFT) represent pivotal paradigms for equipping Large Language Model (LLM) agents with tool-use capabilities without the prohibitive costs of full-model retraining. Unlike supervised fine-tuning which permanently alters model weights, ICL leverages the model’s pre-existing knowledge through strategic prompt construction, enabling rapid adaptation to novel APIs and dynamic environments. A primary challenge in this domain is the efficient management of context windows when dealing with extensive tool documentation. [23] addresses this by transforming diverse and redundant API documentation into unified, concise tool instructions, significantly reducing token consumption while enhancing the model’s ability to select and utilize tools correctly. Similarly, [93] employs an iterative grounding mechanism, retrieving relevant API references to refine code completion predictions, thereby mitigating the hallucination of non-existent functions in repository-level generation tasks.

Beyond static prompting, advanced frameworks treat prompt engineering as an optimizable process. [94] introduces a compiler-like approach where LLM pipelines are defined as computational graphs with declarative modules, automatically optimizing prompts and few-shot exemplars to maximize specific metrics without manual intervention. This aligns with findings from [95], which suggests that base models can achieve alignment comparable to instruction-tuned counterparts through carefully curated in-context examples, challenging the necessity of extensive weight updates for certain agent behaviors. For scenarios involving massive toolsets where context limits preclude loading all descriptions, [10] proposes a hybrid approach: it represents each tool as a unique token (“toolken”) with a learnable embedding while keeping the base LLM frozen. This allows the model to scale to arbitrary numbers of tools and leverage extensive demonstration data for embedding learning, bridging the gap between the flexibility of ICL and the performance of fine-tuning.

Parameter-efficient adaptation further extends these capabilities through modular updates. Techniques such as Low-Rank Adaptation (LoRA) enable the injection of trainable parameters into frozen models, facilitating task-specific alignment with minimal computational overhead. [96]

demonstrates the efficacy of LoRA in enabling open-source models to utilize multimodal tools via self-instructed datasets, achieving robust zero-shot generalization to unseen tools. Similarly, [25] utilizes conditional masking and fine-tuning on small, on-device models to master software API interactions, outperforming larger generalist models in specific function-calling benchmarks. The emergence of automated agent design, as seen in [97], pushes this boundary by treating agentic workflows themselves as learnable parameters, where meta-agents discover optimal prompting strategies and tool compositions programmatically.

Despite these advancements, significant trade-offs remain. While ICL offers immediate adaptability, its performance is highly sensitive to the quality and relevance of retrieved context, often struggling with complex, multi-step reasoning compared to fine-tuned counterparts [98]. Conversely, PEFT methods like [99] provide a balance, offering improved specialization over ICL while maintaining the base model’s general capabilities, yet they still require curated data for adapter training. Future research must address the scalability of these methods in open-world settings, particularly in developing robust retrieval mechanisms that can dynamically construct optimal in-context demonstrations for never-before-seen tools, effectively closing the loop between documentation retrieval and execution reliability.

4.4 Autonomous Self-Improvement and Tool Creation

Building on the parameter-efficient adaptation and automated workflow design discussed previously, autonomous self-improvement and tool creation represent the frontier of agent evolution, shifting the paradigm from static tool consumption to dynamic capability synthesis. In this regime, agents transcend pre-defined API boundaries by diagnosing execution failures, generating novel software modules, and integrating these artifacts into a growing skill library. This iterative loop of experience acquisition, refinement, and abstraction allows agents to address capability gaps autonomously, effectively bootstrapping their own competence without continuous human intervention.

A primary mechanism for this evolution is autonomous tool synthesis, where agents act as software developers to create executable code snippets that function as new tools. Frameworks like [35] demonstrate this by collecting specific code solutions for training examples, abstracting them into reusable snippets, and retrieving them to solve complex tasks, thereby expanding the action space beyond general-purpose APIs. Similarly, [100] establishes an ever-growing skill library of executable code in Minecraft, utilizing an iterative prompting mechanism that incorporates environment feedback and self-verification to refine programs. This approach ensures that learned skills are temporally extended, interpretable, and compositional, compounding the agent’s abilities while mitigating catastrophic forgetting. The efficacy of such systems relies on the agent’s ability to treat code not merely as output but as an actionable medium for self-expansion, as evidenced by [27], which argues that unified code action spaces facilitate dynamic revision and self-debugging more effectively than rigid JSON schemas.

Parallel to tool synthesis is the refinement of behavioral policies through iterative experience management. Agents increasingly employ memory mechanisms to store and optimize past interactions, moving beyond static datasets to dynamic repositories of successful and failed trajectories. [101] introduces patterns for refining experiences based on nearest or cumulative batches, utilizing heuristic elimination to prioritize high-quality, frequently used strategies. This mirrors the [76] approach, which induces reusable workflows from past examples to guide future generations, significantly improving success rates in long-horizon web navigation tasks. Furthermore, self-improvement is accelerated through recursive introspection and critique. [102] fine-tunes models to detect and correct

mistakes over multiple turns, treating single-turn prompting as a multi-turn Markov decision process. This is complemented by [103], where specialized critic models provide dense feedback on code quality, enabling agents to identify and repair errors that would otherwise persist in self-generated tools.

However, significant challenges remain in ensuring the reliability and safety of self-evolving systems. The risk of hallucination during tool creation necessitates robust verification loops, often achieved through symbolic execution or sandboxed environments, as seen in [67] and [104]. Moreover, the trade-off between exploration and exploitation becomes critical; agents must balance the computational cost of synthesizing new tools against the efficiency of retrieving existing ones. [105] addresses this by consolidating investigated trajectories into simplified pipelines, reducing API calls by up to 80% while maintaining performance. This tension sets the stage for the subsequent discussion on optimizing these trade-offs for cost-efficient deployment. Future research must focus on formalizing the stability of these self-improvement loops, preventing reward hacking, and developing universal standards for tool representation that allow seamless integration of agent-created modules across diverse ecosystems. As agents move toward [97], the ability to not only use tools but to architect the very frameworks of their own intelligence will define the next generation of autonomous systems.

4.5 Optimization of Exploration-Exploitation and Cost-Efficiency

The optimization of exploration-exploitation trade-offs and cost-efficiency represents a critical frontier in deploying scalable tool-learning agents. While extensive exploration is necessary for acquiring novel skills and adapting to dynamic environments, it incurs substantial computational overhead and latency, particularly when relying on large language models (LLMs) for every decision step. Consequently, recent methodologies have shifted towards adaptive frameworks that dynamically balance the acquisition of new knowledge with the exploitation of established policies to minimize resource consumption.

A primary strategy for enhancing cost-efficiency involves learning optimal routing mechanisms that determine when to invoke expensive LLM queries versus utilizing lightweight, cached policies. The concept of “compute-optimal” scaling suggests that allocating test-time computation adaptively based on prompt difficulty can yield performance superior to uniformly scaling model parameters [106]. Building on this, router models trained on preference data have been developed to dynamically select between strong and weak LLMs, significantly reducing inference costs without compromising response quality [107]. Furthermore, frameworks like [108] demonstrate that task-specific small language models, fine-tuned on high-quality function-calling datasets, can surpass larger generalist models in specific domains while operating entirely on edge devices, thereby eliminating cloud dependency and latency.

To guide exploration efficiently, researchers have integrated prior knowledge to constrain the search space and accelerate convergence. Language-guided exploration methods, such as ELLM, leverage the semantic understanding of LLMs to propose human-meaningful goals, shaping the reward function to prioritize relevant states over random novelty [86]. This approach mitigates the sample inefficiency inherent in tabula rasa reinforcement learning by aligning agent curiosity with abstract world knowledge. Similarly, the DECKARD agent utilizes LLMs to hypothesize an Abstract World Model (AWM) during a “Dream” phase, which is then verified through interaction, effectively blending noisy internet-scale information with grounded environmental dynamics to increase sample efficiency by an order of magnitude [109].

The tension between exploration and exploitation is also formalized in iterative refinement tasks,

such as code repair, where agents must decide whether to exploit a currently promising solution or explore alternative trajectories. This scenario is effectively modeled as an arm-acquiring bandit problem, where Thompson Sampling is employed to balance refining the best-performing program against exploring less-tested candidates, resulting in higher success rates with fewer LLM calls [110]. Moreover, agents like Agent-Pro evolve their policies through depth-first search and policy-level reflection, allowing them to learn from both successful and unsuccessful trajectories to optimize long-term payoffs in complex interactive environments [71].

Despite these advances, challenges remain in preventing agents from optimizing proxy rewards at the expense of safety or correctness, a phenomenon akin to wireheading in embedded agents [111]. Future research must therefore focus on developing robust evaluation protocols that detect reward hacking while maintaining the efficiency gains of adaptive exploration strategies. The integration of formal verification and symbolic constraints into the exploration loop offers a promising direction to ensure that cost-efficient learning does not compromise the reliability of autonomous agents.

5 Evaluation Ecosystems, Benchmarks, and Safety Protocols

5.1 Comprehensive Benchmarking Frameworks for Agent Proficiency

The evaluation of tool-learning agents has rapidly evolved from static question-answering metrics to dynamic, multi-turn frameworks that assess an agent’s capacity to plan, execute, and adapt within interactive environments. This shift necessitates benchmarks that capture the complexity of real-world workflows, where success depends not merely on correct tool selection but on maintaining state consistency and recovering from execution errors. Early efforts like [112] established a multi-dimensional baseline by testing LLMs across eight distinct environments, revealing significant disparities between commercial and open-source models in long-horizon reasoning and instruction following. Similarly, [113] introduced game-based scenarios to isolate specific cognitive capabilities such as spatial reasoning and planning ahead, providing a granular roadmap for identifying architectural gaps.

A critical trend in contemporary benchmarking is the move toward domain-specific rigor and fine-grained capability decomposition. Rather than relying on binary success rates, modern frameworks dissect the agent lifecycle into intent comprehension, tool selection, parameter grounding, and syntax validity. [114] exemplifies this approach by evaluating agents across seven real-world scenarios and five dimensional metrics, uncovering counter-intuitive findings where increased model size does not necessarily correlate with improved tool-learning proficiency. In the realm of data analysis, [115] addresses the challenge of evaluating open-ended tasks by converting questions into closed-form formats for automatic verification, while [38] expands the scope to include tool creation and planning without pre-defined toolsets, thereby testing an agent’s adaptability to novel constraints. Furthermore, [116] contributes to this landscape by offering a self-instruct dataset with strict format controls to evaluate nested and multi-tool calling capabilities.

The fidelity of the evaluation environment itself has become a primary differentiator. High-fidelity simulators now replicate full operating systems and application ecosystems to test generalist agents. [117] provides a scalable, real computer environment supporting task setup and execution-based evaluation across Ubuntu, Windows, and macOS, highlighting severe deficiencies in current multi-modal agents regarding GUI grounding. Complementing this, [118] introduces a controllable world of apps and people, utilizing state-based unit tests to verify complex, interactive code generation while detecting collateral damage. For mobile-specific interactions, [119] and [120] offer specialized environments that challenge agents with cross-application cooperation and visually grounded tasks,

respectively. These environments underscore that robust agent proficiency requires seamless integration of visual perception and action execution, a gap further explored by [121] which targets large multimodal models as visual foundation agents.

Despite these advances, significant challenges remain in standardizing evaluation protocols and ensuring reliability. [31] identified limitations in previous evaluation protocols that artificially inflated pass rates, proposing revised benchmarks to better reflect practical application scenarios. To address inconsistency, [122] introduced the $pass^k$ metric to evaluate agent reliability over multiple trials, revealing that even state-of-the-art models struggle to consistently follow domain-specific rules. Additionally, [123] advocates for analytical evaluation boards that track incremental progress rather than final outcomes, offering deeper insights into failure modes during multi-turn interactions. Looking forward, the field must converge on unified metrics that balance efficiency, cost, and safety. The emergence of self-evolving benchmarks [124] suggests a future where evaluation dynamicity matches the adaptability of the agents themselves, ensuring that proficiency assessments remain relevant as agents master increasingly complex tool ecosystems.

5.2 Adversarial Robustness and Security Vulnerability Analysis

While the preceding section established rigorous benchmarks for assessing agent proficiency, these metrics often overlook the profound security vulnerabilities unique to tool-learning agents. The security posture of these systems extends beyond traditional language model alignment, introducing risks arising from the dynamic coupling of generative reasoning with executable external environments. Unlike static text generation, the function-calling interface creates a mutable attack surface where untrusted data returned by tools can hijack agent trajectories, a phenomenon extensively documented in indirect prompt injection (IPI) attacks. Empirical evaluations reveal that agents are highly susceptible to having their goals overwritten by malicious instructions embedded within tool observations, leading to unauthorized data exfiltration or harmful action execution [125; 126]. This vulnerability is exacerbated in multi-agent ecosystems, where compromised nodes can propagate manipulated knowledge or toxic instructions across the network, creating persistent security risks that survive individual interaction sessions [127].

A critical vector for exploitation involves the function-calling schema itself, which adversaries leverage to bypass safety guardrails through specialized “jailbreak function” attacks. Research demonstrates that automated frameworks utilizing tree-of-thought reasoning can efficiently generate semantic jailbreaks that coerce models into ignoring safety filters during parameter grounding or tool selection phases [128; 129]. These attacks are particularly potent because they exploit the model’s imperative to follow structural constraints over semantic safety rules. Furthermore, the supply chain of tool learning presents severe risks; backdoor attacks such as [126] illustrate how fine-tuning on poisoned datasets can embed triggers that activate malicious behaviors only when specific environmental conditions are met, effectively turning benign agents into dormant threats. Similarly, [126] highlights the fragility of retrieval-augmented generation (RAG) systems, showing that poisoning long-term memory or knowledge bases can force agents to retrieve and execute malicious demonstrations with high probability, all without modifying the underlying model weights.

The operational reliability of agents is also threatened by malfunction amplification, where adversaries induce infinite loops or resource exhaustion by triggering repetitive, irrelevant action sequences. Benchmarks like [37] quantify this instability, revealing that even state-of-the-art models suffer significant performance degradation when confronted with noisy or adversarial environmental inputs, with success rates dropping precipitously under medium to heavy noise conditions. In

high-stakes domains, these vulnerabilities translate to tangible risks; for instance, LLM agents have been shown to autonomously exploit one-day and zero-day cybersecurity vulnerabilities in real-world systems when provided with tool access, outperforming traditional vulnerability scanners [130; 131]. This capability underscores the dual-use nature of advanced agents, where the same reasoning mechanisms used for defensive automation can be weaponized for offensive operations.

Current defensive paradigms often struggle to balance robustness with utility. While frameworks like [132] propose constitutional safeguards integrated into planning stages, and multi-agent defense systems like [133] employ role-playing critics to filter harmful responses, these measures frequently fail against adaptive attacks that evolve faster than static rule sets. Moreover, the non-deterministic nature of LLM reasoning complicates the formal verification of agent behavior, making it difficult to guarantee safety across all possible execution paths. These limitations in static defense and verification necessitate a paradigm shift toward more robust architectural solutions. Consequently, future research must pivot towards dynamic, runtime verification mechanisms and neuro-symbolic approaches that can mathematically constrain action spaces while preserving the flexibility required for complex tool interaction, setting the stage for the detailed survey of defensive architectures and behavioral alignment strategies discussed in the following section.

5.3 Safety Alignment, Guardrails, and Risk Mitigation Strategies

The deployment of autonomous agents capable of executing external functions necessitates a paradigm shift from static content safety to dynamic behavioral alignment. This subsection surveys defensive architectures that mitigate risks inherent in tool learning, categorizing them into internal alignment strategies, external guardrail systems, and structural constraints on the action space. A critical challenge lies in preserving safety alignment during downstream adaptation; benign fine-tuning on agent-specific trajectories can inadvertently erode safety guardrails, leading to the emergence of harmful behaviors even when training data appears safe [134]. To counteract this, recent frameworks propose constitutive safety mechanisms that embed explicit rules directly into the agent’s planning lifecycle. For instance, TrustAgent introduces a tripartite strategy involving pre-planning safety injection, in-planning constraint enforcement, and post-planning inspection to prevent dangerous tool invocations before execution [132].

Complementing internal alignment, external guardrail systems operate as modular monitoring layers independent of the core model, offering robustness against jailbreaks and adversarial inputs without requiring retraining. Multi-agent defense frameworks, such as AutoDefense, leverage role-specialized agents to collaboratively filter harmful responses, demonstrating that distributed verification significantly enhances robustness against sophisticated jailbreak attacks compared to single-model filtering [133]. Similarly, intention analysis techniques have proven effective as inference-only defenses, triggering the model’s inherent self-correction capabilities to neutralize stealthy attacks while maintaining helpfulness [135]. However, these software-level defenses face limitations against indirect prompt injections embedded within tool outputs, where malicious instructions from external environments (e.g., emails or websites) can hijack agent trajectories [125]. Consequently, comprehensive risk assessment benchmarks like ToolSword and R-Judge have emerged to evaluate agent safety across the entire interaction lifecycle, revealing that even state-of-the-art models remain vulnerable to risky tool usage and harmful feedback loops [136; 137].

Beyond monitoring, structural constraints on the parameter and action space provide a foundational layer of risk mitigation. By restricting the agent’s operational scope through type-constrained decoding and policy-based access controls, systems can prevent syntactic errors and block unauthorized

API calls based on user privileges. The concept of “formal-LLM” further advances this by integrating formal languages, such as automata, to supervise plan generation, ensuring that agent actions strictly adhere to user-defined safety constraints and logical validity [138]. Yet, the threat landscape extends to supply chain vulnerabilities, where backdoor attacks can embed triggers within the agent’s reasoning process, activating malicious tool usage only under specific environmental conditions [139; 140]. Furthermore, the dual-use nature of advanced agents poses severe risks in high-stakes domains; research indicates that coordinated teams of LLM agents can autonomously exploit zero-day vulnerabilities, highlighting the urgent need for rigorous capability controls [131]. Future research must therefore converge on neuro-symbolic hybrid approaches that combine the flexibility of language models with the verifiability of symbolic planners, ensuring that autonomy does not come at the expense of predictability and safety.

5.4 Metrics for Reliability, Efficiency, and Human-Agent Alignment

Transitioning from the safety foundations and structural constraints discussed above, assessing the practical utility of tool-learning agents necessitates a paradigm shift from binary success rates to multidimensional metrics that capture reliability, efficiency, and alignment. While traditional benchmarks often rely on final task completion, such measures obscure critical failures in intermediate reasoning steps and resource utilization. To address this, fine-grained evaluation frameworks like [123] introduce progress rate metrics that quantify incremental advancements, offering deeper interpretability into agent behaviors than holistic success scores. Furthermore, the reliability of agents in dynamic environments requires metrics that account for consistency across multiple trials. The τ -bench proposes the $pass^k$ metric to evaluate behavioral stability, revealing that even state-of-the-art models often exhibit high variance in rule-following capabilities across repeated executions [122]. This inconsistency underscores the need for robustness quantification that extends beyond single-shot performance to include resilience against environmental noise and adversarial perturbations.

Beyond reliability, efficiency in tool learning is not merely a function of execution speed but involves a complex trade-off between computational cost, token consumption, and action optimality. Agents must balance the exploration of novel tool combinations with the exploitation of known strategies to minimize latency and financial overhead. Frameworks like [23] demonstrate that refining tool instructions can significantly reduce token usage while maintaining performance, suggesting that metric definitions must incorporate “cost-per-success” ratios rather than raw accuracy alone. Additionally, the concept of step efficiency—minimizing redundant or unnecessary tool calls—is critical for scaling agents to long-horizon tasks. Research indicates that agents often suffer from “action loops” or redundant queries, necessitating metrics that penalize excessive interaction turns relative to the optimal path length [141].

Human-agent alignment represents the most challenging dimension to quantify, requiring metrics that bridge the gap between technical correctness and user intent satisfaction. Standard alignment metrics often fail to capture the nuance of implicit user goals or the appropriateness of agent autonomy. The [115] addresses this by converting open-ended data analysis queries into closed-form evaluations, yet this approach may overlook the qualitative aspect of human utility. To better measure alignment, recent works propose evaluating agents on their ability to clarify vague instructions and recover missing information proactively, as seen in the Intention-in-Interaction (IN3) benchmark which assesses an agent’s capacity to refine user goals before execution [142]. Moreover, safety alignment must be evaluated not just as a static constraint but as a dynamic property; [137] introduces benchmarks specifically for risk awareness, measuring an agent’s proficiency in identifying

and halting unsafe operations within multi-turn interactions.

Looking forward, the field must converge on unified evaluation standards that integrate these disparate dimensions. Emerging trends point toward self-evolving benchmarks that adapt to agent capabilities to prevent saturation, as proposed in [124]. Ultimately, a comprehensive metric suite must jointly optimize for high success rates, low computational overhead, and strict adherence to human values, ensuring that tool-learning agents are not only capable but also trustworthy and economically viable for real-world deployment.

6 Domain-Specific Applications and Emerging Frontiers

6.1 Autonomous Software Engineering and Code-Centric Agents

Autonomous Software Engineering represents a paradigm shift where Large Language Model (LLM) agents transition from passive code synthesizers to active engineers capable of managing the full software development lifecycle. This transformation is predicated on the agent’s ability to invoke external tools—compilers, debuggers, version control systems, and static analyzers—via function calling, thereby grounding abstract reasoning in executable reality. Unlike traditional models that generate isolated snippets, code-centric agents operate within a feedback loop of planning, execution, and self-correction, effectively mitigating the “near miss syndrome” where generated code semantically resembles correctness but fails unit tests due to minor syntactic or logical imperfections [68].

A critical advancement in this domain is the development of specialized Agent-Computer Interfaces (ACIs) that optimize the interaction between LLMs and development environments. [67] demonstrates that tailoring interfaces to agent capabilities significantly enhances repository navigation and code editing, achieving state-of-the-art performance on benchmarks like SWE-bench. Furthermore, the integration of executable code as a unified action space, as proposed in CodeAct, allows agents to dynamically compose tools and self-debug through multi-turn interactions, outperforming rigid JSON-based function calling by enabling flexible control flows [27]. This capability is essential for repository-level generation, where agents must resolve cross-file dependencies and undefined variables. Approaches like ToolGen address these challenges by integrating autocompletion tools directly into the generation process, triggering tool calls at specific token markers to ensure static validity and dependency coverage [143]. Similarly, De-Hallucinator employs iterative grounding to retrieve project-specific API references, significantly reducing the hallucination of non-existent APIs [93].

The complexity of real-world software engineering often necessitates multi-agent collaboration, mirroring human organizational structures. Frameworks such as Magis deploy specialized agents—including managers, repository custodians, and quality assurance engineers—to collaboratively resolve GitHub issues, leveraging role specialization to overcome the context limitations of single models [144]. Extending this, MapCoder replicates the human program synthesis cycle through distinct agents for planning, generation, and debugging, achieving superior pass@1 rates on competitive programming benchmarks [145]. These collaborative systems are further enhanced by frameworks like AutoAgents, which dynamically generate expert agents based on task requirements, and Cross-Team Collaboration (CTC), which orchestrates multiple development teams to explore diverse solution paths simultaneously [18; 146].

Despite these advances, significant challenges remain regarding cost efficiency and the scalability of autonomous workflows. The iterative nature of self-repair can incur substantial token costs,

prompting the development of cost-efficient strategies like CigaR, which optimizes prompt settings to minimize expenditure while maintaining repair efficacy [74]. Moreover, recent analyses suggest that simplistic, non-agentic approaches focusing on precise localization and repair can sometimes outperform complex autonomous agents, highlighting the need for rigorous evaluation and architectural simplification [147]. Future directions point toward self-evolving ecosystems where agents not only utilize tools but also autonomously generate and refine their own toolsets, as seen in frameworks enabling symbolic learning and inter-task evolution [148; 105]. Ultimately, the convergence of robust tool learning, multi-agent orchestration, and efficient self-correction mechanisms promises to realize fully autonomous software engineering systems capable of navigating the intricacies of large-scale codebases.

6.2 Embodied Intelligence and Robotic Control Systems

Parallel to the transformation of software engineering agents, the integration of Large Language Models (LLMs) into embodied intelligence represents a paradigmatic shift from static policy execution to dynamic, language-grounded reasoning in physical worlds. This subsection examines the architectural evolution wherein LLMs function not merely as high-level planners but as direct interfaces for low-level robotic control, translating abstract natural language instructions into precise motion primitives and control signals. A primary challenge in this domain is the “grounding gap,” where the semantic richness of language must be mapped to the continuous, high-dimensional action space of physical agents. Early approaches addressed this by treating LLMs as symbolic planners that sequence pre-defined skills. For instance, [8] demonstrated that LLMs could generate Python code to compose vision-based perception outputs with control primitive APIs, enabling spatial-geometric reasoning and the synthesis of reactive policies without fine-tuning. Similarly, [149] utilized programmatic prompt structures to constrain plan generation within situated environments, ensuring that generated action sequences remain executable given specific robot capabilities.

However, reliance on fixed skill libraries limits adaptability in open-world scenarios, mirroring the constraints faced by early code-centric agents. Consequently, recent research has pivoted toward dynamic skill synthesis and reward shaping, where LLMs generate the objectives or logic for low-level controllers rather than just sequencing actions. [87] introduced a human-level reward design algorithm where LLMs perform evolutionary optimization over reward code, enabling the acquisition of complex dexterous skills like pen spinning that exceed human-engineered baselines. Complementing this, [90] proposed utilizing LLMs to define reward parameters optimized by Model Predictive Control (MPC), effectively bridging high-level corrections with low-level actuation. This transition allows agents to handle novel tasks by synthesizing control logic on the fly, as seen in [150], which maps multi-modal instructions to comprehensive perception-planning-action loops via generated Python programs.

A critical frontier in this evolution is the fusion of vision-language models (VLMs) with planning to achieve true multimodal grounding, a necessity shared with the GUI automation domain discussed subsequently. Purely text-based planners often hallucinate feasible actions due to a lack of perceptual awareness. To mitigate this, [151] leveraged VLMs to compose 3D value maps from language instructions, enabling zero-shot trajectory synthesis that is robust to dynamic perturbations. Furthermore, [43] argued for inherently grounded, unified multimodal systems where VLMs directly integrate perceptual data into the reasoning process, surpassing LLM-only planners in long-horizon manipulation tasks. The necessity of closed-loop feedback is further emphasized by [6], which showed that incorporating environmental feedback—such as success detection and scene descriptions—into the LLM’s context forms an “inner monologue” that significantly improves in-

struction completion rates in real-world settings.

Despite these advances, challenges persist regarding sample efficiency and long-horizon coherence. Methods like [89] attempt to bridge abstract language and learned low-level control by using motion planning to guide reinforcement learning from scratch, achieving high success rates without pre-determined skills. Additionally, [152] addresses the limitation of stationary skill sets by introducing agents that continuously grow their robot skill libraries through self-guided exploration and memory distillation. Looking forward, the field is moving toward fully autonomous, self-evolving embodied agents. Frameworks like [100] exemplify this trend, employing automatic curricula and ever-growing skill libraries to master open-ended environments like Minecraft without human intervention. The future of embodied intelligence lies in hybrid neuro-symbolic architectures that combine the generative flexibility of LLMs with the geometric precision of classical control, ensuring safe, reliable, and adaptable interaction in unstructured physical worlds, thereby establishing a foundational logic that extends into the digital GUI automation landscape.

6.3 Multimodal Interaction and Graphical User Interface Automation

The convergence of Multimodal Large Language Models (MLLMs) and tool learning has precipitated a paradigm shift in Graphical User Interface (GUI) automation, transitioning agents from rigid, script-based executors to adaptive entities capable of perceiving, reasoning, and acting within complex digital ecosystems. This domain is characterized by a fundamental dichotomy in interaction mechanisms: API-first abstraction versus pixel-level direct manipulation. The API-first approach prioritizes reliability and semantic grounding by leveraging backend function calls, thereby bypassing the fragility of visual element detection. Frameworks such as [23] exemplify this by distilling diverse tool documentation into concise, unified instructions, enabling agents to interact with enterprise software through standardized interfaces rather than volatile UI elements. Similarly, [24] demonstrates that fine-tuned models connected to massive API repositories can surpass general-purpose LLMs in generating accurate function calls, significantly mitigating hallucination in tool usage. However, this approach is inherently limited by the availability of exposed APIs, rendering it ineffective for closed-source applications or legacy systems.

Conversely, direct UI manipulation agents operate on a “see-and-click” paradigm, treating the screen as a visual canvas to be interpreted and acted upon. This necessitates robust visual grounding, where agents must map natural language intents to specific coordinates or UI elements within dynamic screenshots. [153] introduces a simplified action space mimicking human interactions like tapping and swiping, allowing agents to learn navigation strategies through autonomous exploration or human demonstration without requiring system-level backend access. Advanced frameworks like [154] and [155] enhance this capability by integrating comprehensive environment perception, combining screenshots with detailed layout structures and historical action contexts to improve action prediction accuracy. Despite these advances, purely vision-based approaches face significant challenges in scalability and inference efficiency, often struggling with long-horizon tasks due to context window limitations and the high computational cost of processing high-resolution visual tokens [156].

To address these limitations, emerging research advocates for hybrid architectures that synthesize the precision of API calls with the universality of visual grounding. [157] expands MLLM capabilities by maintaining a skill repository of pre-trained vision models, activating relevant tools based on user input to fulfill real-world tasks that require both visual understanding and external tool execution. Furthermore, the complexity of modern operating systems, which often require

cross-application cooperation, has spurred the development of multi-agent collaborative frameworks. [158] illustrates this by decomposing mobile operation tasks among planning, decision, and reflection agents, effectively managing task progress navigation and correcting erroneous operations through iterative self-reflection. This modular decomposition is critical for handling the vast and dynamic action spaces found in general-purpose operating systems, where single-agent architectures frequently fail to maintain coherence [119].

Looking forward, the field is moving towards the creation of generalist computer agents capable of operating across heterogeneous environments. Benchmarks like [117] and [120] are establishing rigorous evaluation standards for open-ended tasks in real computer environments, revealing that current agents still lag significantly behind human performance in GUI grounding and operational knowledge. Future advancements will likely depend on the integration of temporal understanding to handle dynamic GUI content, as static image analysis proves insufficient for complex, multi-step workflows [159]. Additionally, the development of efficient, on-device models such as [25] suggests a trend towards low-latency, privacy-preserving agents that can execute function calls locally. Ultimately, achieving robust multimodal interaction requires not only improved visual encoders but also sophisticated reasoning mechanisms that can seamlessly switch between high-level semantic planning and low-level pixel-based execution, bridging the gap between abstract intent and concrete digital action.

6.4 Scalable Multi-Agent Collaboration and Organizational Architectures

Bridging the gap between the generalist computer agents and multi-agent collaborations outlined previously, the transition from single-agent tool usage to scalable multi-agent collaboration represents a paradigm shift in addressing complex, multi-domain problems that exceed the cognitive and contextual limits of individual Large Language Models (LLMs). This subsection analyzes architectural patterns that enable dynamic agent generation, recursive delegation, and robust conflict resolution within distributed organizational structures. Central to this evolution is the move away from static, predefined agent teams toward adaptive ecosystems where roles and communication topologies are synthesized on demand. Frameworks such as [18] demonstrate the efficacy of dynamically generating specialized agents based on task semantics, coupling role definition with planning to enhance solution coherence. Similarly, [160] introduces inference-time agent selection and early-stopping mechanisms, optimizing team composition through an unsupervised “Agent Importance Score” to balance performance and computational cost.

A critical architectural trend involves the formalization of organizational hierarchies to manage communication overhead and prevent information redundancy. Inspired by human corporate structures, prompt-based organizational frameworks impose designated leadership and specialized labor roles, significantly improving team efficiency in embodied tasks [161]. This structured approach contrasts with flat, peer-to-peer collaboration models like [162], which leverage round-table consensus and confidence-weighted voting among diverse models to mitigate individual hallucinations. While consensus mechanisms enhance reliability, they often incur higher latency; thus, hybrid approaches such as [16] employ a central controller to orchestrate distinct labor agents, each focusing on specific action types, thereby streamlining the execution flow. Furthermore, the concept of “Cross-Team Collaboration” [146] extends this by enabling parallel development chains, allowing multiple agent teams to explore diverse solution paths simultaneously before synthesizing optimal outcomes, effectively addressing the local optima trap inherent in single-chain reasoning.

Scalability in these systems is increasingly achieved through recursive delegation and self-organizing

principles. The [97] framework illustrates how agents can automatically multiply in response to problem complexity, ensuring that the code volume managed by each agent remains constant while the total system capacity scales indefinitely. This mirrors the recursive decomposition strategies seen in [29], where sub-tasks are broken down as needed, but applied here across distinct agent boundaries. In large-scale code generation and optimization, such dynamic scaling allows for the management of expansive action spaces that would otherwise overwhelm a single context window. However, these distributed architectures introduce significant challenges in conflict resolution and state consistency. Without rigorous coordination protocols, agents may engage in contradictory actions or propagate manipulated knowledge throughout the network [127]. To counteract this, emerging research integrates formal verification and guardian agents to audit inter-agent communications and enforce safety constitutions [132].

Looking forward, the convergence of multi-agent systems with automated system design suggests a future where agents not only collaborate but also autonomously architect their own interaction protocols, setting the stage for the self-evolving ecosystems discussed in the next section. The [97] paradigm posits that meta-agents can iteratively discover superior collaboration topologies and communication strategies in code, transcending human-designed limitations. Additionally, the integration of operating system-level abstractions, as proposed in [163], aims to provide foundational support for concurrent agent execution, context switching, and resource allocation, treating agents as first-class processes within a unified computational environment. Ultimately, the maturation of scalable multi-agent architectures hinges on balancing the flexibility of dynamic role assignment with the stability of formalized governance mechanisms, ensuring that collective intelligence emerges without succumbing to coordination failures or security vulnerabilities, while paving the way for agents that can continuously synthesize and optimize their own functional capabilities.

6.5 Emerging Frontiers in Tool Synthesis and Self-Evolving Ecosystems

The frontier of tool learning is rapidly shifting from static reliance on pre-defined API repositories toward dynamic ecosystems where agents autonomously synthesize, optimize, and evolve their own functional capabilities. This paradigm transition addresses the inherent limitations of fixed toolsets, which often fail to cover edge cases or adapt to novel environmental dynamics. A primary direction in this evolution is automated tool discovery and black-box probing, where agents actively interact with unknown interfaces to infer input-output schemas and generate documentation without human intervention. Frameworks such as [164] demonstrate that agents can act as multi-tool learners by utilizing black-box probing methods to actively discover and document tool usages, effectively teaching themselves to master new tools beyond manually demonstrated sets. Complementing this, [165] proposes a training-free method where code language models curate reusable, high-level functions by iteratively generating, using, growing, and trimming a toolbox, resulting in simpler solutions with higher accuracy across diverse domains.

Parallel to tool synthesis is the emergence of self-evolving data pipelines designed to bootstrap agent capabilities through autonomous experience generation. Instead of relying on costly human curation, agents are increasingly tasked with generating high-quality, diverse function-calling training data through simulated scenarios and mutual verification. [36] illustrates this by leveraging an automated pipeline to synthesize verifiable datasets, enabling even small parameter models to achieve state-of-the-art performance on function-calling benchmarks. Furthermore, [166] introduces self-guided methodologies where models autonomously discern high-quality samples based on an Instruction-Following Difficulty metric, significantly enhancing training efficiency. This capacity for self-improvement extends to the optimization of the agents’ own operational instructions; [23]

presents a framework that transforms redundant tool documentation into unified, concise instructions, reducing token consumption while improving utilization performance.

The concept of self-evolution also encompasses the structural optimization of agentic systems themselves, moving beyond simple prompt engineering to automated architectural design. [97] formulates the Automated Design of Agentic Systems (ADAS) research area, where meta-agents iteratively program novel agent designs in code, discovering architectures that outperform hand-designed counterparts across multiple domains. Similarly, [148] introduces agent symbolic learning, a framework that treats agents as symbolic networks optimized via natural language simulacrum of back-propagation, allowing deployed agents to update their policies and tools autonomously. This iterative refinement is further supported by mechanisms like [101], which enables agents to refine experiences cumulatively across task batches, prioritizing high-quality interactions to manage the experience space effectively.

Despite these advancements, significant challenges remain in ensuring the verifiability and safety of self-synthesized tools. The risk of error propagation in autonomous loops necessitates robust verification layers, as seen in the hierarchical validation stages of [36]. Moreover, the computational cost of continuous evolution requires efficient strategies for balancing exploration and exploitation. Future research must therefore focus on developing universal verification protocols for synthesized tools and establishing theoretical bounds on the convergence of self-evolving agent ecosystems. By integrating active probing, automated data synthesis, and structural self-optimization, the field is poised to transition from agents that merely use tools to autonomous entities that continuously expand their own cognitive and operational frontiers.

7 Conclusion

The trajectory of tool learning has fundamentally redefined the capabilities of Large Language Model (LLM) agents, marking a paradigm shift from static text generators to dynamic entities capable of orchestrating complex interactions with external environments. This survey has synthesized the evolution from monolithic, fine-tuned approaches to modular, multi-agent architectures that decouple planning, memory, and execution [2; 167; 17]. While early frameworks like [168] demonstrated the viability of self-supervised tool acquisition, contemporary systems increasingly leverage neuro-symbolic hybridization to ensure logical consistency and robustness in long-horizon tasks [48; 169]. The integration of structured decoding mechanisms and syntax-constrained generation has further mitigated hallucination risks, enabling agents to adhere strictly to API schemas while maintaining the flexibility of natural language reasoning [170; 136].

Despite these advancements, significant bottlenecks impede the widespread deployment of autonomous agents in high-stakes domains. A primary limitation lies in the fragility of current planning mechanisms when confronted with dynamic environmental feedback; agents often struggle to adaptively refine strategies without compounding errors, a challenge addressed partially by closed-loop refinement frameworks like [65] and [126]. Furthermore, the scalability of tool repositories presents a critical hurdle. As the number of available functions expands, retrieval accuracy degrades, necessitating advanced semantic matching and intent-aware selection protocols to prevent context window overflow and decision paralysis [82; 40; 32]. Security remains another pervasive concern; recent analyses reveal that agents are susceptible to prompt injection, jailbreak attacks via function calling, and the amplification of malicious feedback, underscoring the inadequacy of current safety guardrails in open-ended interaction loops [136; 171; 172]. Benchmarks such as [112] and [173] have exposed substantial performance gaps between proprietary and open-source models,

particularly in tasks requiring stateful dependency tracking and multi-turn coherence [35; 174].

Looking forward, the field must pivot toward self-evolving ecosystems where agents autonomously synthesize, validate, and optimize their own toolsets, moving beyond passive consumption of pre-defined APIs [175; 164]. Future research should prioritize the development of universal tool representation languages that facilitate zero-shot generalization across heterogeneous domains, reducing the reliance on task-specific fine-tuning [82]. The integration of causal reasoning mechanisms is essential to distinguish correlation from causation in environmental feedback, thereby enhancing the agent’s ability to perform counterfactual reasoning and robust error recovery [75]. Moreover, the emergence of automated design frameworks, such as [97], suggests a promising direction where meta-agents iteratively discover optimal architectural patterns, potentially surpassing human-engineered solutions. Ultimately, realizing the full potential of tool-learning agents requires a holistic approach that balances autonomy with rigorous safety constraints, ensuring that as agents gain the capacity to execute irreversible real-world actions, their alignment with human intent and ethical standards remains uncompromised [132; 172].

References

- [1] Tool Learning with Foundation Models
- [2] A Survey on Large Language Model based Autonomous Agents
- [3] TALM Tool Augmented Language Models
- [4] What Are Tools Anyway A Survey from the Language Model Perspective
- [5] Granite-Function Calling Model: Introducing Function Calling Abilities via Multi-task Learning of Granular Tasks
- [6] Inner Monologue Embodied Reasoning through Planning with Language Models
- [7] Emergent autonomous scientific research capabilities of large language models
- [8] Code as Policies Language Model Programs for Embodied Control
- [9] Large Multimodal Agents A Survey
- [10] ToolkenGPT Augmenting Frozen Language Models with Massive Tools via Tool Embeddings
- [11] Learning to Use Tools via Cooperative and Interactive Agents
- [12] AI Chains Transparent and Controllable Human-AI Interaction by Chaining Large Language Model Prompts
- [13] Grounding Large Language Models in Interactive Environments with Online Reinforcement Learning
- [14] MetaTool Benchmark for Large Language Models Deciding Whether to Use Tools and Which to Use
- [15] AgentTuning Enabling Generalized Agent Abilities for LLMs
- [16] BOLAA Benchmarking and Orchestrating LLM-augmented Autonomous Agents
- [17] Small LLMs Are Weak Tool Learners A Multi-LLM Agent
- [18] AutoAgents A Framework for Automatic Agent Generation

- [19] Language Agents as Optimizable Graphs
- [20] ToolChain Efficient Action Space Navigation in Large Language Models with A Search
- [21] Self-Organized Agents A LLM Multi-Agent Framework toward Ultra Large-Scale Code Generation and Optimization
- [22] AgentLite A Lightweight Library for Building and Advancing Task-Oriented LLM Agent System
- [23] EASYTOOL Enhancing LLM-based Agents with Concise Tool Instruction
- [24] Gorilla Large Language Model Connected with Massive APIs
- [25] Octopus On-device language model for function calling of software APIs
- [26] Copiloting the Copilots Fusing Large Language Models with Completion Engines for Automated Program Repair
- [27] Executable Code Actions Elicit Better LLM Agents
- [28] PromptChainer Chaining Large Language Model Prompts through Visual Programming
- [29] Canvil Designerly Adaptation for LLM-Powered User Experiences
- [30] MLLM-Tool A Multimodal Large Language Model For Tool Agent Learning
- [31] AnyTool Self-Reflective, Hierarchical Agents for Large-Scale API Calls
- [32] GeckoOpt LLM System Efficiency via Intent-Based Tool Selection
- [33] ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities
- [34] TOOLVERIFIER Generalization to New Tools via Self-Verification
- [35] CRAFT Customizing LLMs by Creating and Retrieving from Specialized Toolsets
- [36] APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets
- [37] RoTBench A Multi-Level Benchmark for Evaluating the Robustness of Large Language Models in Tool Learning
- [38] Planning, Creation, Usage Benchmarking LLMs for Comprehensive Tool Utilization in Real-World Complex Scenarios
- [39] Translating Natural Language to Planning Goals with Large-Language Models
- [40] Efficient Tool Use with Chain-of-Abstraction Reasoning
- [41] Robots That Ask For Help Uncertainty Alignment for Large Language Model Planners
- [42] Interactive Task and Concept Learning from Natural Language Instructions and GUI Demonstrations
- [43] Look Before You Leap Unveiling the Power of GPT-4V in Robotic Vision-Language Planning
- [44] RepairAgent An Autonomous, LLM-Based Agent for Program Repair
- [45] KnowAgent Knowledge-Augmented Planning for LLM-Based Agents

- [46] Agent Planning with World Knowledge Model
- [47] On the Planning Abilities of Large Language Models A Critical Investigation
- [48] LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks
- [49] LLMs Still Can't Plan; Can LRMs? A Preliminary Evaluation of OpenAI's o1 on PlanBench
- [50] Can LLMs Follow Simple Rules
- [51] Data Interpreter An LLM Agent For Data Science
- [52] Language-Augmented Symbolic Planner for Open-World Task Planning
- [53] Tool-Planner: Dynamic Solution Tree Planning for Large Language Model with Tool Clustering
- [54] Large Language Models as Commonsense Knowledge for Large-Scale Task Planning
- [55] Reasoning with Language Model is Planning with World Model
- [56] When is Tree Search Useful for LLM Planning It Depends on the Discriminator
- [57] Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models
- [58] LLM is Like a Box of Chocolates the Non-determinism of ChatGPT in Code Generation
- [59] Alphazero-like Tree-Search can Guide Large Language Model Decoding and Training
- [60] Tree Search for Language Model Agents
- [61] Advancing LLM Reasoning Generalists with Preference Trees
- [62] Toward Self-Improvement of LLMs via Imagination, Searching, and Criticizing
- [63] Efficient Exploration for LLMs
- [64] ReAct Synergizing Reasoning and Acting in Language Models
- [65] AdaPlanner Adaptive Planning from Feedback with Language Models
- [66] Reason for Future, Act for Now A Principled Framework for Autonomous LLM Agents with Provable Sample Efficiency
- [67] SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering
- [68] Fully Autonomous Programming with Large Language Models
- [69] LEVER Learning to Verify Language-to-Code Generation with Execution
- [70] Retroformer Retrospective Large Language Agents with Policy Gradient Optimization
- [71] Agent-Pro Learning to Evolve via Policy-Level Reflection and Optimization
- [72] Making Language Models Better Tool Learners with Execution Feedback
- [73] Trial and Error Exploration-Based Trajectory Optimization for LLM Agents
- [74] CigaR Cost-efficient Program Repair with LLMs
- [75] Shaking the foundations delusions in sequence models for interaction and control
- [76] Agent Workflow Memory

- [77] RAP Retrieval-Augmented Planning with Contextual Memory for Multimodal LLM Agents
- [78] ExpeL LLM Agents Are Experiential Learners
- [79] WorldCoder, a Model-Based LLM Agent Building World Models by Writing Code and Interacting with the Environment
- [80] AutoGuide Automated Generation and Selection of State-Aware Guidelines for Large Language Model Agents
- [81] LLMs in the Imaginarium Tool Learning through Simulated Trial and Error
- [82] ToolAlpaca Generalized Tool Learning for Language Models with 3000 Simulated Cases
- [83] AbsInstruct Eliciting Abstraction Ability from LLMs through Explanation Tuning with Plausibility Estimation
- [84] Agent Q: Advanced Reasoning and Learning for Autonomous AI Agents
- [85] Motif Intrinsic Motivation from Artificial Intelligence Feedback
- [86] Guiding Pretraining in Reinforcement Learning with Large Language Models
- [87] Eureka Human-Level Reward Design via Coding Large Language Models
- [88] Training LLMs to Better Self-Debug and Explain Code
- [89] Plan-Seq-Learn: Language Model Guided RL for Solving Long Horizon Robotics Tasks
- [90] Language to Rewards for Robotic Skill Synthesis
- [91] Collaborating with language models for embodied reasoning
- [92] True Knowledge Comes from Practice Aligning LLMs with Embodied Environments via Reinforcement Learning
- [93] De-Hallucinator Iterative Grounding for LLM-Based Code Completion
- [94] DSPy Compiling Declarative Language Model Calls into Self-Improving Pipelines
- [95] The Unlocking Spell on Base LLMs Rethinking Alignment via In-Context Learning
- [96] GPT4Tools Teaching Large Language Model to Use Tools via Self-instruction
- [97] Automated Design of Agentic Systems
- [98] Chain of Thoughtlessness? An Analysis of CoT in Planning
- [99] LLM-Adapters An Adapter Family for Parameter-Efficient Fine-Tuning of Large Language Models
- [100] Voyager An Open-Ended Embodied Agent with Large Language Models
- [101] Iterative Experience Refinement of Software-Developing Agents
- [102] Recursive Introspection: Teaching Language Model Agents How to Self-Improve
- [103] LLM Critics Help Catch LLM Bugs
- [104] OpenDevin: An Open Platform for AI Software Developers as Generalist Agents
- [105] Investigate-Consolidate-Exploit A General Strategy for Inter-Task Agent Self-Evolution

- [106] Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters
- [107] RouteLLM: Learning to Route LLMs with Preference Data
- [108] TinyAgent: Function Calling at the Edge
- [109] Do Embodied Agents Dream of Pixelated Sheep Embodied Decision Making using Language Guided World Modelling
- [110] Code Repair with LLMs gives an Exploration-Exploitation Tradeoff
- [111] Categorizing Wireheading in Partially Embedded Agents
- [112] AgentBench Evaluating LLMs as Agents
- [113] SmartPlay A Benchmark for LLMs as Intelligent Agents
- [114] ToolEyes Fine-Grained Evaluation for Tool Learning Capabilities of Large Language Models in Real-world Scenarios
- [115] InfiAgent-DABench Evaluating Agents on Data Analysis Tasks
- [116] Seal-Tools: Self-Instruct Tool Learning Dataset for Agent Tuning and Detailed Benchmark
- [117] OSWorld Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments
- [118] AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents
- [119] Understanding the Weakness of Large Language Model Agents within a Complex Android Environment
- [120] VisualWebArena Evaluating Multimodal Agents on Realistic Visual Web Tasks
- [121] VisualAgentBench: Towards Large Multimodal Models as Visual Foundation Agents
- [122] τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains
- [123] AgentBoard An Analytical Evaluation Board of Multi-turn LLM Agents
- [124] Benchmark Self-Evolving A Multi-Agent Framework for Dynamic LLM Evaluation
- [125] InjecAgent Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents
- [126] More Agents Is All You Need
- [127] Flooding Spread of Manipulated Knowledge in LLM-Based Multi-Agent Communities
- [128] Tree of Attacks Jailbreaking Black-Box LLMs Automatically
- [129] Jailbreaking Black Box Large Language Models in Twenty Queries
- [130] LLM Agents can Autonomously Exploit One-day Vulnerabilities
- [131] Teams of LLM Agents can Exploit Zero-Day Vulnerabilities
- [132] TrustAgent Towards Safe and Trustworthy LLM-based Agents through Agent Constitution

- [133] AutoDefense Multi-Agent LLM Defense against Jailbreak Attacks
- [134] Keeping LLMs Aligned After Fine-tuning The Crucial Role of Prompt Templates
- [135] Intention Analysis Makes LLMs A Good Jailbreak Defender
- [136] ToolSword Unveiling Safety Issues of Large Language Models in Tool Learning Across Three Stages
- [137] R-Judge Benchmarking Safety Risk Awareness for LLM Agents
- [138] Formal-LLM Integrating Formal Language and Natural Language for Controllable LLM-based Agents
- [139] Watch Out for Your Agents! Investigating Backdoor Threats to LLM-Based Agents
- [140] BadAgent: Inserting and Activating Backdoor Attacks in LLM Agents
- [141] On the Tool Manipulation Capability of Open-source Large Language Models
- [142] Tell Me More! Towards Implicit User Intention Understanding of Language Model Driven Agents
- [143] Teaching Code LLMs to Use Autocompletion Tools in Repository-Level Code Generation
- [144] MAGIS LLM-Based Multi-Agent Framework for GitHub Issue Resolution
- [145] MapCoder: Multi-Agent Code Generation for Competitive Problem Solving
- [146] Multi-Agent Software Development through Cross-Team Collaboration
- [147] Agentless: Demystifying LLM-based Software Engineering Agents
- [148] Symbolic Learning Enables Self-Evolving Agents
- [149] ProgPrompt Generating Situated Robot Task Plans using Large Language Models
- [150] Instruct2Act Mapping Multi-modality Instructions to Robotic Actions with Large Language Model
- [151] VoxPoser Composable 3D Value Maps for Robotic Manipulation with Language Models
- [152] Lifelong Robot Library Learning: Bootstrapping Composable and Generalizable Skills for Embodied Control with Language Models
- [153] AppAgent Multimodal Agents as Smartphone Users
- [154] ScreenAgent A Vision Language Model-driven Computer Control Agent
- [155] Comprehensive Cognitive LLM Agent for Smartphone GUI Automation
- [156] On the Effects of Data Scale on Computer Control Agents
- [157] LLaVA-Plus Learning to Use Tools for Creating Multimodal Agents
- [158] Mobile-Agent-v2: Mobile Device Operation Assistant with Effective Navigation via Multi-Agent Collaboration
- [159] GUI-WORLD: A Dataset for GUI-oriented Multimodal LLM-based Agents

- [160] Dynamic LLM-Agent Network An LLM-agent Collaboration Framework with Agent Team Optimization
- [161] Embodied LLM Agents Learn to Cooperate in Organized Teams
- [162] ReConcile Round-Table Conference Improves Reasoning via Consensus among Diverse LLMs
- [163] AIOS LLM Agent Operating System
- [164] Chain of Tools: Large Language Model is an Automatic Multi-tool Learner
- [165] TroVE Inducing Verifiable and Efficient Toolboxes for Solving Programmatic Tasks
- [166] From Quantity to Quality Boosting LLM Performance with Self-Guided Data Selection for Instruction Tuning
- [167] The Rise and Potential of Large Language Model Based Agents A Survey
- [168] Toolformer Language Models Can Teach Themselves to Use Tools
- [169] On the Prospects of Incorporating Large Language Models (LLMs) in Automated Planning and Scheduling (APS)
- [170] Efficiently Programming Large Language Models using SGLang
- [171] Prioritizing Safeguarding Over Autonomy Risks of LLM Agents for Science
- [172] Application of LLM Agents in Recruitment A Novel Framework for Resume Screening
- [173] StableToolBench Towards Stable Large-Scale Benchmarking on Tool Learning of Large Language Models
- [174] m&m’s A Benchmark to Evaluate Tool-Use for multi-step multi-modal Tasks
- [175] Large Language Models as Tool Makers